Task 3: Using Git Stash to Manage Conflicting Changes

Step-by-step lab solution with commands, expected output, and conflict handling

Objective: Temporarily save uncommitted changes with Git stash, continue working on another task, and restore the saved changes.

Step 1 — Create a new Git repository

```
mkdir git-stash-exercise
cd git-stash-exercise
git init -b main
```

Configure your Git identity if it has not already been configured:

```
git config --global user.name "Student"
git config --global user.email "student@gmail.com"
```

Step 2 — Create app.txt and make the initial commit

```
echo "Application version 1" > app.txt
git add app.txt
git commit -m "Initial application version"
```

The file contains:

```
Application version 1
```

Step 3 — Make an uncommitted change

```
echo "Feature version A" >> app.txt
cat app.txt
git status
```

The file now contains:

```
Application version 1
Feature version A
```

Git reports that app.txt has been modified but not committed.

Step 4 — Save changes using git stash

```
git stash
git status
cat app.txt
```

Expected status:

```
On branch main
nothing to commit, working tree clean
```

Git saves the uncommitted change and restores the working directory to the last committed state. The file now contains only:

```
Application version 1
```

Step 5 — Make a different change and commit it

```
echo "Feature version B" >> app.txt
git add app.txt
git commit -m "Update application version"
```

The file contains:

```
Application version 1
Feature version B
```

There are now two commits: the initial application version and the update. Feature version A remains saved in the stash.

Step 6 — Restore the stashed changes

```
git stash pop
git status
cat app.txt
```

Important: With these exact changes, Git will normally apply the stash successfully without a conflict because the appended lines can usually be combined automatically.

After a successful application, the file will typically contain:

```
Application version 1
Feature version B
Feature version A
```

The restored changes are usually unstaged. A conflict is possible in other overlapping-change situations, but it is not guaranteed by this task's exact steps.

Step 7 — Inspect and handle a conflict if one occurs

If Git reports a conflict, inspect the file:

```
cat app.txt
```

Example conflict markers:

```
Application version 1
<<<<<<< Updated upstream
Feature version B
=======
Feature version A
>>>>>>> Stashed changes
```

The markers identify the current committed version and the stashed version. Edit the file, remove all conflict markers, retain Feature version B, and add the required new feature line.

Required final content:

```
Application version 1
Feature version B
New feature under development
```

If no conflict occurs, edit the successfully merged file to obtain this same required final content.

Step 8 — Stage and commit the final changes

```
git add app.txt
git commit -m "Resolve stash conflict"
```

This commit records the final contents of app.txt. If no conflict occurred, a message such as `Complete stashed feature` would describe the action more accurately, but the requested message can still be used.

Step 9 — Examine the final repository state

```
git status
git log --oneline
git stash list
```

Expected status:

```
On branch main
nothing to commit, working tree clean
```

Example commit history (hashes will differ):

```
c3d4e5f Resolve stash conflict
b2c3d4e Update application version
a1b2c3d Initial application version
```

After a successful `git stash pop`, the stash entry is normally removed, so `git stash list` should be empty. If the pop operation conflicts, Git generally keeps the stash entry.

If a stash entry remains, verify that its changes have been recovered and committed before removing it:

```
git stash list
git stash drop
```

Caution: Use `git stash drop` only after confirming that the saved changes are no longer needed.

Final verification

The final app.txt should contain:

```
Application version 1
Feature version B
New feature under development
```

Command	Purpose
git stash	Temporarily saves uncommitted changes.
git stash pop	Applies saved changes and normally removes the stash entry.
git status	Shows working-directory and staging-area state.
git log --oneline	Displays concise commit history.
git stash list	Lists saved stash entries.
git add app.txt	Stages the final file.
git commit -m "Resolve stash conflict"	Records the final changes.

Key takeaway: Git stash lets you temporarily set aside unfinished work while you make another change. When you restore the saved work, Git may merge it automatically or report a conflict if changes overlap.